

Google Cloud 기반 Antigravity AI Agent 구축 및 MVP 완성 과정

PROJECT 1

시장·경쟁사 정보 수집 & 추천 자동화 Agent

Simplified Prompt Edition

Git + GitHub + GitHub Actions + GitHub Pages

핵심 흐름

공개 웹/RSS → 수집 → 정제 → 기업 맞춤 추천 → 웹 리포트 → GitHub → 자동 실행 → GitHub Pages

Version 1.1 | 2026.09 | 초급자용 간소화 버전

1. 이번 개정판의 방향

첫 번째 프로젝트에서는 Antigravity 사용법, Git/GitHub, 자동화와 배포를 한꺼번에 처음 접한다. 따라서 기존 교재의 세분화된 프롬프트를 일부 묶고, 반복되는 규칙은 한 번만 제시하도록 간소화했다.

구분	기존	간소화 버전
프롬프트 수	세부 단계별로 다수	핵심 10 개
제약조건	각 프롬프트마다 반복	공통 규칙으로 통합
GitHub	인증/초기화/생성/Push 분리	인증은 사전, 업로드는 하나로 통합
배포	Secret/Actions/Pages 분리	하나의 자동배포 프롬프트로 통합

중요: 기능 범위는 줄이지 않는다. 학생이 입력해야 하는 프롬프트의 수와 반복 문장만 줄인다.

2. 최종 결과

- 시장·경쟁사·정책·지원사업 정보를 공개 웹/RSS 에서 수집한다.
- 수집 실패 시 제공되는 800 행 fallback 데이터를 사용한다.
- 중복·결측·노이즈를 정제하고 기업 Profile 에 맞게 추천한다.
- HTML/CSS/JavaScript 기반 정적 Dashboard 를 생성한다.
- GitHub 에 프로젝트를 올리고 GitHub Actions 로 자동 실행한다.
- 결과 Dashboard 를 GitHub Pages 에서 확인한다.

최종 예시

<https://USERNAME.github.io/project1-market-agent/>

3. 제공 데이터와 목표 구조

기존 Project 1 Starter ZIP 을 그대로 사용한다. 핵심 데이터는 data/fallback/fallback_market_news.csv 이며 총 800 행으로 구성되어 있다.

```
project1-market-agent/  
├─ config/company_profile.yaml  
├─ data/fallback/fallback_market_news.csv  
├─ data/raw/  
├─ data/processed/  
├─ src/  
├─ docs/  
├─ logs/  
├─ .github/workflows/  
├─ .env.example  
├─ .gitignore  
├─ requirements.txt  
└─ README.md
```

4. 모든 프롬프트에 적용하는 공통 규칙

아래 규칙은 이후 Prompt 01~10 에 공통으로 적용한다. 따라서 매번 같은 Constraints 를 길게 반복하지 않는다.

[Common Rules]

- 작업 전에 현재 프로젝트 구조와 관련 파일을 먼저 확인한다.
- 기존 폴더 구조를 불필요하게 변경하지 않는다.
- 기존 기능과 중복되는 코드를 새로 만들지 않는다.
- 파일명과 함수명은 영어로 작성한다.
- 필요한 경우에만 새 라이브러리를 추가한다.
- .env, API Key, 인증정보는 GitHub 에 올리지 않는다.
- 접근 제한, 로그인, CAPTCHA 를 우회하지 않는다.
- 작업 후 반드시 실행하고 결과를 검증한다.
- 완료 후 수정 파일, 결과, 오류, 남은 문제를 보고한다.

수강생은 이후 프롬프트를 입력할 때 위 Common Rules 가 계속 적용된다고 AntigraVity 에 알려주면 된다.

5. 사전 준비

필요 도구: Python 3.12, Git, GitHub CLI(gh), AntigraVity. GitHub 계정 로그인과 브라우저 승인만 수강생이 직접 수행한다.

```
python --version
git --version
gh --version
gh auth status
```

Git 또는 GitHub CLI 가 없다면 AntigraVity 에게 설치 방법을 안내받아 설치한다. GitHub 인증은 gh auth login 을 사용한다.

6. Prompt 01 - 환경 점검과 GitHub 인증

첫 프롬프트에서 환경 점검과 GitHub 인증을 함께 처리한다. 인증 창이 열리면 수강생이 직접 브라우저 승인을 한다.

Antigravity 입력 프롬프트

[Context]

Project 1 을 시작한다. Python 3.12 기반이며 이후 GitHub 에 올리고 GitHub Pages 로 배포할 예정이다. Windows 환경이다.

[Goal]

Python, Git, GitHub CLI 상태와 GitHub 인증 상태를 한 번에 확인한다.

[Scope]

Python / pip / git / gh

[Execution]

1. 각 버전을 확인한다.
2. 없는 도구가 있으면 Windows 설치 방법을 안내한다.
3. gh auth status 를 확인한다.
4. 로그인되어 있지 않으면 gh auth login 을 실행하고 브라우저 인증 단계에서 내가 직접 승인하도록 안내한다.
5. 인증 후 다시 상태를 확인한다.

[Verification]

Python, pip, git, gh 가 모두 실행되고 GitHub 계정이 인증되어야 한다.

[Report]

버전, 로그인 상태, 다음 단계 진행 가능 여부를 정리한다.

완료 확인

- ☐ Python/Git/gh 정상
- ☐ GitHub 인증 완료

7. Prompt 02 - 프로젝트 구조와 기업 Profile 준비

프로젝트 생성과 기업 Profile 정의를 하나로 묶어 초기 진입 단계를 줄인다.

Antigravity 입력 프롬프트

[Context]

Starter ZIP 의 데이터로 project1-market-agent 를 시작한다. 시장/경쟁사 수집 기준은 config/company_profile.yaml 에 정의한다.

[Goal]

프로젝트 구조를 준비하고 기업 Profile 을 점검한다.

[Scope]

config/, data/, src/, docs/, logs/, .github/, requirements.txt, README.md

[Execution]

1. 현재 폴더를 분석한다.
2. 필요한 기본 폴더가 없으면 생성한다.
3. fallback CSV 와 company_profile.yaml 을 올바른 위치에 둔다.
4. company_profile.yaml 의 company_name, business_area, products, target_market, competitors, interest_keywords, funding_keywords 를 확인한다.
5. 경쟁사 최소 3 개, 관심 키워드 최소 5 개가 되도록 예시 값을 보완한다.

[Verification]

프로젝트 트리와 YAML 문법을 확인하고 fallback CSV 가 존재해야 한다.

[Report]

프로젝트 구조와 기업/경쟁사/검색 키워드를 요약한다.

완료 확인

- ☐ fallback CSV 존재
- ☐ company_profile.yaml 정상

8. Prompt 03 - 공개 Source 조사

Antigravity 입력 프롬프트

[Context]

company_profile.yaml 기준으로 시장, 경쟁사, 정책, 정부지원, 기술동향을 수집하려 한다.

[Goal]

실제로 접근 가능한 공개 Source 를 정하고 수집 계획을 만든다.

[Scope]

공개 RSS, 공개 뉴스/정책 페이지, 공개 API 후보

[Execution]

1. 기업 Profile 을 읽는다.
2. 최소 5 개의 Source 후보를 찾는다.
3. RSS → requests → 공개 API → 필요한 경우 Playwright 순으로 수집 방법을 정한다.
4. 접근 가능 여부와 예상 수집량을 확인한다.
5. 최소 200 건 확보 가능성을 평가한다.

[Output]

logs/source_plan.md

[Verification]

로그인 필요한 Source 없이 3 종류 이상의 정보가 포함되어야 한다.

완료 확인

- ☐ Source 5 개 이상
- ☐ 200 건 이상 수집 가능성 확인

9. Prompt 04 - Crawler 와 fallback 처리

Antigravity 입력 프롬프트

[Context]

logs/source_plan.md 가 준비되어 있고 data/fallback/fallback_market_news.csv 에는 수업용 fallback 데이터가 있다.

[Goal]

시장·경쟁사·정책 정보를 수집하는 crawler.py 를 만들고 실행한다.

[Scope]

src/crawler.py, requirements.txt, data/raw/, logs/

[Execution]

1. source_plan.md 를 기준으로 crawler.py 를 구현한다.
2. 공개 RSS/페이지를 수집하여 data/raw/crawled_market_news.csv 로 저장한다.

3. Source 하나가 실패해도 전체 작업은 계속한다.
4. 200 건 미만이면 대체 Source 를 시도한다.
5. 그래도 부족하면 fallback 데이터를 합친다.
6. live/fallback 은 data_origin 컬럼으로 구분한다.

[Verification]

최종 200 건 이상, title/source_url/source_name/data_origin 컬럼이 존재하고 실행 오류가 없어야 한다.

[Report]

실제 수집 건수, fallback 건수, 실패 Source 를 정리한다.

완료 확인

- ☐ raw CSV 200 건 이상
- ☐ live/fallback 구분

10. Prompt 05 - 데이터 정제

Antigravity 입력 프롬프트

[Context]

data/raw/crawled_market_news.csv 에 수집 데이터가 있다. 중복, 결측, HTML 노이즈와 낱자 오류가 포함될 수 있다.

[Goal]

분석 가능한 정제 데이터를 만든다.

[Scope]

src/cleaner.py, data/raw/, data/processed/

[Execution]

1. 원본 데이터의 컬럼/타입을 확인한다.
2. HTML 태그와 불필요한 공백을 제거한다.
3. title 결측과 지나치게 짧은 데이터를 처리한다.
4. 낱자를 정규화한다.
5. 완전 중복, 제목 중복, URL 중복을 제거한다.
6. 결과를 data/processed/cleaned_market_news.csv 로 저장한다.

[Verification]

중복 article_id/URL 이 없고 빈 title 이 없어야 한다.

[Report]

원본 건수, 최종 건수, 중복/결측 제거 수를 출력한다.

완료 확인

- ☐ 중복 URL 없음
- ☐ 빈 title 없음

11. Prompt 06 - 기업 맞춤 추천과 Dashboard

기존의 추천과 Dashboard 생성을 하나로 묶는다. Gemini API 가 없어도 규칙 기반 방식으로 완주할 수 있다.

Antigravity 입력 프롬프트

[Context]

company_profile.yaml 과 cleaned_market_news.csv 가 준비되어 있다. 결과를 기업 맞춤형으로 추천하고 GitHub Pages 용 정적 Dashboard 로 만들려고 한다.

[Goal]

추천 결과와 웹 리포트를 한 번에 생성한다.

[Scope]

src/recommender.py, src/build_site.py, data/processed/, docs/

[Execution]

1. keyword/company/competitor/funding 관련성 점수를 계산한다.
2. Gemini API Key 가 있으면 기업 관련성·사업 중요도·대응 필요성을 추가 평가하고 추천 이유를 만든다.
3. API Key 가 없으면 규칙 기반 방식만으로 계속 실행한다.
4. 상위 30 개를 recommended_market_news.csv 로 저장한다.
5. TOP 10, category 통계, 추천 이유와 원문 링크를 포함하는 docs/index.html 과 docs/report.json 을 만든다.

[Verification]

상위 30 개 추천, 추천 이유, Source URL 이 존재하고 docs/index.html 이 브라우저에서 정상 표시되어야 한다.

[Report]

TOP 10 제목과 점수, LLM 사용 여부, 웹 리포트 생성 결과를 정리한다.

완료 확인

- ☐ 추천 30 개
- ☐ docs/index.html 정상

12. Prompt 07 - Pipeline 통합과 오류 복구

Antigravity 입력 프롬프트

[Context]

crawler.py, cleaner.py, recommender.py, build_site.py 가 준비되어 있다. 실제 웹 수집은 Timeout, 403, RSS 오류가 발생할 수 있다.

[Goal]

전체 작업을 run_pipeline.py 하나로 실행하고 실패에 대응한다.

[Scope]

src/run_pipeline.py 와 기존 src 모듈, logs/

[Execution]

1. 기존 모듈을 중복 구현하지 말고 순서대로 호출한다.
2. 수집 → 정제 → 추천 → Dashboard 를 실행한다.
3. 각 단계 상태를 SUCCESS/WARNING/FAILED 로 기록한다.
4. URL 오류, Timeout, 403, RSS 파싱 실패, 200 건 미만 상황을 안전하게 처리한다.
5. 수집이 부족하면 fallback 을 사용하고 최종 Dashboard 생성을 계속한다.

[Verification]

python src/run_pipeline.py 한 번으로 raw/cleaned/recommended CSV 와 docs/index.html 이 생성되어야 한다.

[Output]

logs/pipeline.log

완료 확인

- ☐ run_pipeline.py 단일 실행
- ☐ 오류 시 fallback 동작

13. Prompt 08 - Git 초기화와 GitHub 업로드

Antigravity 입력 프롬프트

[Context]

Project 1 기능 테스트가 완료되었고 GitHub CLI 인증도 되어 있다.

[Goal]

현재 프로젝트를 Git Repository 로 만들고 GitHub Public Repository 에 Push 한다.

[Scope]

현재 project1-market-agent 폴더와 현재 GitHub 계정

[Execution]

1. .gitignore 와 민감정보를 확인한다.
2. 필요하면 git init 후 main branch 를 사용한다.
3. 안전한 파일만 add/commit 한다.
4. GitHub 에 project1-market-agent 가 없으면 gh repo create 로 Public Repository 를 만든다.
5. origin 을 연결하고 main 을 Push 한다.

[Verification]

.env/API Key 가 Commit 되지 않았고 GitHub 에서 main branch 와 파일을 확인할 수 있어야 한다.

[Report]

Repository URL, Commit hash, Push 결과를 정리한다.

완료 확인

- ☐ GitHub Repository 생성
- ☐ main Push 성공

14. Prompt 09 - GitHub Actions + Secret + Pages 자동 배포

초급자에게 가장 복잡한 GitHub 자동화 구간을 하나의 프롬프트로 묶되, Secret 실제 값 입력만 수강생이 직접 한다.

Antigravity 입력 프롬프트

[Context]

프로젝트가 GitHub 에 Push 되어 있고 python src/run_pipeline.py 로 전체 실행 가능하다. 결과 정적 사이트는 docs/에 생성 된다.

[Goal]

GitHub Actions 에서 자동 실행하고 GitHub Pages 로 배포한다.

[Scope]

.github/workflows/market-agent.yml, Repository Secrets, GitHub Pages

[Execution]

1. Gemini 를 사용할 경우 GEMINI_API_KEY Secret 존재 여부를 확인한다. 없으면 gh secret set GEMINI_API_KEY 를 실행하고 실제 값 입력은 내가 직접 하도록 안내한다.
2. Workflow 를 생성한다. Trigger 는 main push, workflow_dispatch, 매일 오전 9 시 Asia/Seoul 이다.
3. Workflow 에서 checkout → Python 설정 → 패키지 설치 → run_pipeline.py → docs/index.html 확인 → Pages 배포 순으로 실행한다.
4. Pages 가 비활성화되어 있으면 workflow 방식으로 활성화한다.
5. Workflow 를 한 번 수동 실행하고 완료 상태와 Pages URL 을 확인한다.

[Verification]

Actions 가 성공하고 Pages URL 에서 최신 Dashboard 가 열려야 한다.

[Report]

Workflow 상태, Pages URL, 다음 예약 실행 정보를 정리한다.

완료 확인

- ☐ Actions 성공
- ☐ Pages URL 접근

15. Prompt 10 - 최종 검증

Antigravity 입력 프롬프트

[Context]

Project 1 구현과 GitHub Pages 배포가 완료되었다.

[Goal]

수업 성공 기준을 최종 점검한다.

[Execution]

다음 순서대로 확인한다.

- 데이터 200 건 이상
- 중복/결측 처리
- 기업/경쟁사 맞춤 추천
- 추천 이유와 원문 링크
- python src/run_pipeline.py 단일 실행
- 오류 발생 시 fallback 처리
- .env 와 API Key Git 제외
- GitHub Repository 와 main branch
- GitHub Actions 수동/예약 Trigger
- GitHub Pages 정상 접근

[Verification]

각 항목을 PASS / WARNING / FAIL 로 표시한다.

[Report]

최종 성공 여부, Repository URL, Pages URL, 남은 문제와 개선사항을 표로 정리한다.

완료 확인

- ☐ 전체 항목 PASS/WARNING/FAIL 정리

16. 전체 실습 흐름 요약

단계	프롬프트	핵심 결과
1	환경 + 인증	Python/Git/GitHub 준비
2	프로젝트 + 기업 Profile	폴더 구조와 맞춤 검색 기준
3	Source 조사	수집 계획
4	Crawler	raw CSV + fallback
5	정제	cleaned CSV
6	추천 + Dashboard	recommended CSV + 정적 웹
7	Pipeline + 복구	run_pipeline.py

단계	프롬프트	핵심 결과
8	Git + GitHub	Repository / Push
9	Actions + Pages	자동 실행 / 웹 배포
10	최종 검증	PASS/WARNING/FAIL 보고서

초급자용 Project 1에서는 위 10개 프롬프트까지만 사용하고, GitHub CLI의 세부 옵션이나 Workflow YAML 내부 구조는 결과를 확인하면서 설명하는 정도로 진행한다. 이후 Project 2부터는 수강생이 이 구조화 프롬프트 패턴에 익숙해졌다는 전제로 조금 더 세분화된 프롬프트를 사용할 수 있다.

17. 최종 제출물

- GitHub Repository URL
- GitHub Pages URL
- 정상 실행된 GitHub Actions 기록
- 시장·경쟁사 추천 Dashboard

18. 강사용 운영 포인트

- Prompt 01~02까지는 천천히 진행해 Antigravity의 Context/Goal/Scope 개념을 설명한다.
- Prompt 03~07은 결과 파일을 직접 열어 확인하는 데 시간을 더 배분한다.
- 실시간 크롤링 실패는 수업 실패가 아니다. fallback 전환이 정상 작동하면 성공으로 본다.
- GitHub 인증, Secret 입력처럼 본인 인증이 필요한 부분만 학생이 직접 수행한다.
- GitHub Actions와 Pages의 세부 YAML 문법을 암기시키기보다 자동화 흐름을 이해시키는 데 초점을 둔다.